Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЁТ

по лабораторной работе № 4
по дисциплине

ЛОГИЧЕСКИЕ ОСНОВЫ ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМ

Студенты гр. 421702:

Сайковский А. В.
Мяло В. В.
Вашкевич М. В.

Проверил:

В. П. Ивашенко

Минск 2026

ХОД ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

Вариант 1

Тема: Программирование операций нечёткого вывода и неклассических логик.
Цель: Приобрести навыки разработки и программирования прикладных систем логического вывода, включая системы неклассического логического вывода.
Задание: На основе первой лабораторной работы реализовать обратный нечёткий логический вывод на основе max-min композиции.

Описание работы

Разработана программа на языке Python, выполняющая обратный нечёткий логический вывод: по отношению R и известному следствию $C = B'$ она определяет все посылки A', для которых $A' \circ R = C$. Программа размещена в одном файле `lab4.py`. Чтение базы знаний, построение отношения Гёделя, прямой max-min вывод и форматирование множеств импортируются из `lab1.py`.

Входные данные задаются обычным текстовым файлом в формате первой лабораторной. Отношение можно построить по правилу $A(x) \to B(y)$ или непосредственно задать его строками. Результат представляет собой объединение декартовых областей допустимых значений. Если система несовместна, выводится сообщение об отсутствии решений.

Теоретические сведения

Пусть $X = \{x_1, \ldots, x_n\}$ и $Y = \{y_1, \ldots, y_m\}$ — конечные непустые универсумы. В прямой задаче по известной посылке A' вычисляется заключение C. В обратной задаче известны коэффициенты r_{ij} отношения и степени $c_j = \mu_C(y_j)$, а неизвестны $a_i = \mu_{A'}(x_i)$:

$$\max_i \min(a_i, r_{ij}) = c_j, \qquad j = 1, \ldots, m. \tag{1}$$

Каждая неизвестная и каждый коэффициент принадлежат $[0, 1]$.

Для правила первой лабораторной используется импликация Гёделя:

$$I_G(p,q) = \begin{cases} 1, & p \le q, \\ q, & p > q, \end{cases} \qquad r_{ij} = I_G(\mu_A(x_i), \mu_B(y_j)). \tag{2}$$

Множество B, задающее правило, и известное следствие C могут различаться.

Если $r_{ij} > c_j$, условие $\min(a_i, r_{ij}) \le c_j$ требует $a_i \le c_j$. Если $r_{ij} \le c_j$, ограничение выполняется при любом $a_i \in [0,1]$. Поэтому все решения ограничены сверху вектором

$$u_i = \min_j I_G(r_{ij}, c_j) = \min\big(\{c_j : r_{ij} > c_j\} \cup \{1\}\big). \tag{3}$$

Построенный u удовлетворяет всем верхним ограничениям: $u \circ R \le C$. Если имеется решение a, то $a \le u$. По монотонности композиции $C = a \circ R \le u \circ R$, откуда

$$S \neq \varnothing \quad \Longleftrightarrow \quad u \circ R = C. \tag{4}$$

При выполнении равенства u является наибольшим точным решением. Проверка выполняется непосредственно функцией `max_min_composition()` из первой лабораторной.

Для описания всех решений недостаточно найти только u. При $0 \le a \le u$ каждое положительное уравнение должно иметь хотя бы один индекс-свидетель:

$$J_j = \{i : r_{ij} \ge c_j,\ u_i \ge c_j\}, \qquad \exists i \in J_j :\ a_i \ge c_j.$$

Алгоритм начинает с нулевого нижнего вектора. Для каждого положительного уравнения он перебирает допустимые индексы и поднимает выбранную координату до c_j. Затем удаляются дубли и доминируемые векторы: если $p \le q$, область $[q,u]$ содержится в $[p,u]$, и вектор q не нужен. Нулевые уравнения уже обеспечены верхними границами.

Пусть L — найденные минимальные решения. Любое решение лежит

над одним из них, а все точки между l и u также дают C. Поэтому

$$S = \bigcup_{l \in L} [l, u], \qquad [l, u] = \prod_{i} [l_i, u_i]. \tag{5}$$

Области могут пересекаться; их объединение не всегда является одним декартовым произведением. Теоретическая основа изложена в практикуме БГУИР и работе Э. Санчеса из списка источников.

Формат базы знаний

По умолчанию программа читает `lab4/input.txt`. Другой файл передаётся первым аргументом: `python lab4.py lab4/examples/input3.txt`. Используются те же регулярные выражения и функция `read_knowledge_base()` из первой лабораторной. Пустые строки и комментарии, начинающиеся с `#`, допускаются. Имя `C` зарезервировано для известного следствия.

```
<факт> ::= <имя> = { <список пар> }
<список пар> ::= <пара> | <пара>, <список пар>
<пара> ::= < <элемент>, <степень> >
<правило> ::= <имя>(<переменная>) ~> <имя>(<переменная>)
<имя> ::= <буква> | <буква><символ>
<символ> ::= <буква> | <цифра>
<буква> ::= A | ... | Z | a | ... | z
<цифра> ::= 0 | ... | 9
<степень> ::= число от 0 до 1
```

Если в файле есть правило, его посылка и заключение задают отношение Гёделя, а отдельный факт C — наблюдаемое следствие:

```
A = {<x1, 0.3>, <x2, 0.8>}
B = {<y1, 0.4>, <y2, 0.7>}
C = {<y1, 0.4>, <y2, 0.7>}

A(x) ~> B(y)
```

Если правила нет, каждый факт, кроме C, задаёт строку матрицы R. Имена этих фактов определяют порядок неизвестных координат. Например, отношение из двух единиц и следствие 0.5 записываются так:

```
X1 = {<y1, 1.0>}
X2 = {<y1, 1.0>}
C = {<y1, 0.5>}
```

В этом случае неизвестны степени для строк `X1` и `X2`. Каждая строка отношения должна содержать тот же набор элементов, что и C. Проверяются формат строк, наличие следствия, число правил, ссылки на факты, уникальность имён и элементов, а также диапазон степеней. Значения хранятся как `float`, как в первой лабораторной; операции min и max выбирают из введённых машинных значений, не выполняя арифметического накопления ошибки.

Структуры данных и функции программы

Как и в первой лабораторной, нечёткое множество представляется списком пар `(element, degree)`, а отношение — списком троек `(x, y, degree)`. Словарь `upper` хранит верхнюю границу для каждой неизвестной координаты. Набор нижних границ хранится как множество кортежей, что сразу исключает дубли.

- `read_problem(filename)` — вызывает чтение базы из первой лабы, проверяет данные и получает отношение и следствие;
- `find_minimal_solutions(...)` — перебирает свидетелей максимумов и оставляет минимальные нижние векторы;
- `inverse_fuzzy_inference(...)` — находит верхнюю границу, проверяет существование решения прямой композицией и вызывает поиск минимальных решений;
- `format_result(...)` — формирует запись множества решений как объединения областей;
- `main()` — выбирает входной файл, выполняет вывод и печатает результат.

Из `lab1.py` напрямую импортируются `read_knowledge_base()`, `build_godel_relation()`, `max_min_composition()`, `format_fuzzy_set()` и шаблоны фактов и правил. Их реализация не дублируется. Число вариантов может расти экспоненциально. При превышении 10 000 кандидатов программа завершает обработку с сообщением об ошибке, не выдавая частичный результат за полный.

Схемы алгоритмов программы

Редактируемые исходники схем сохранены в `lab4/figures/drawio/`. Они используют те же элементы и формат draw.io, что и схемы первой лабораторной. В отчёт подключены соответствующие PNG. После изменения схемы её можно снова экспортировать в PNG под тем же именем и пересобрать отчёт. Вспомогательное форматирование результата описано текстом.

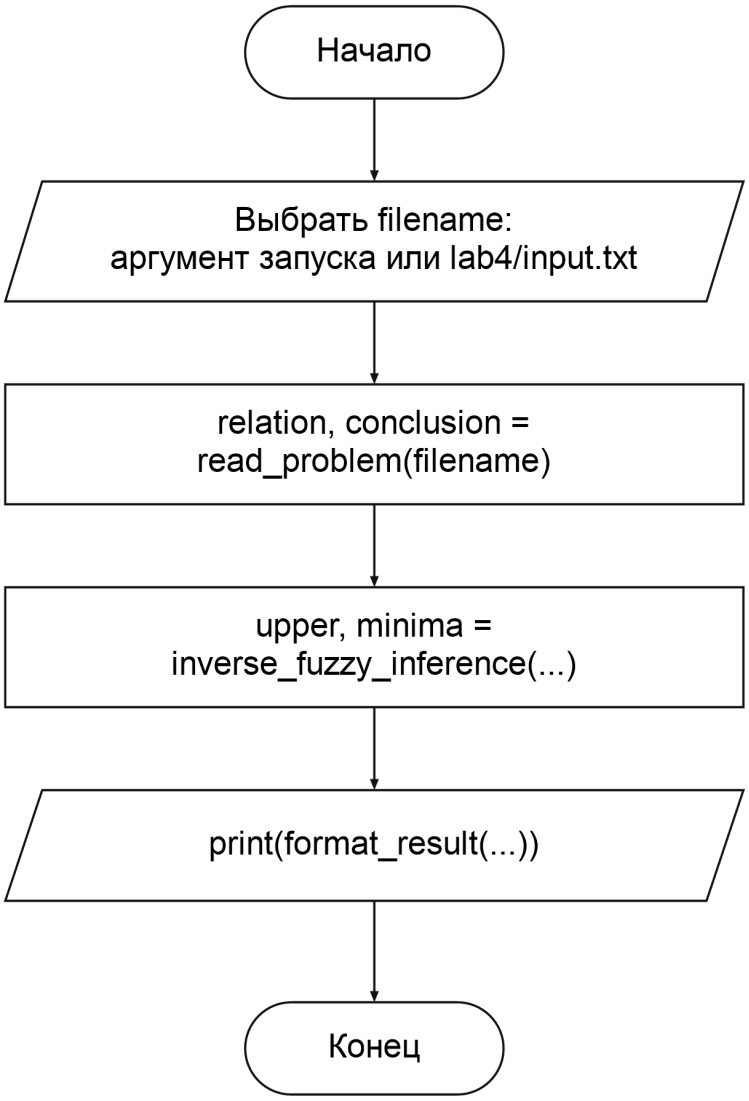


Рис. 1 Схема функции `main()`

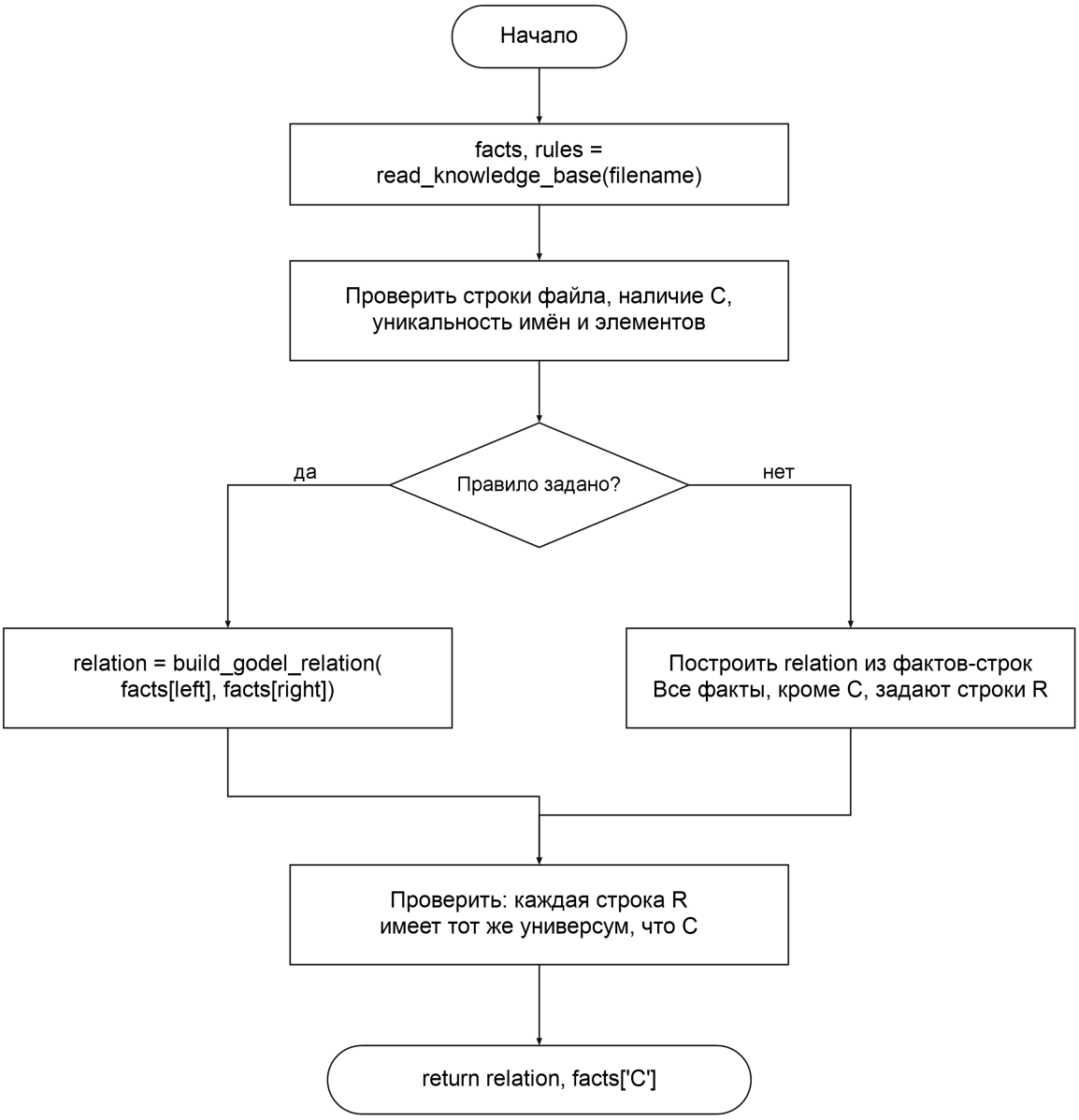


Рис. 2 Схема функции `read_problem()`

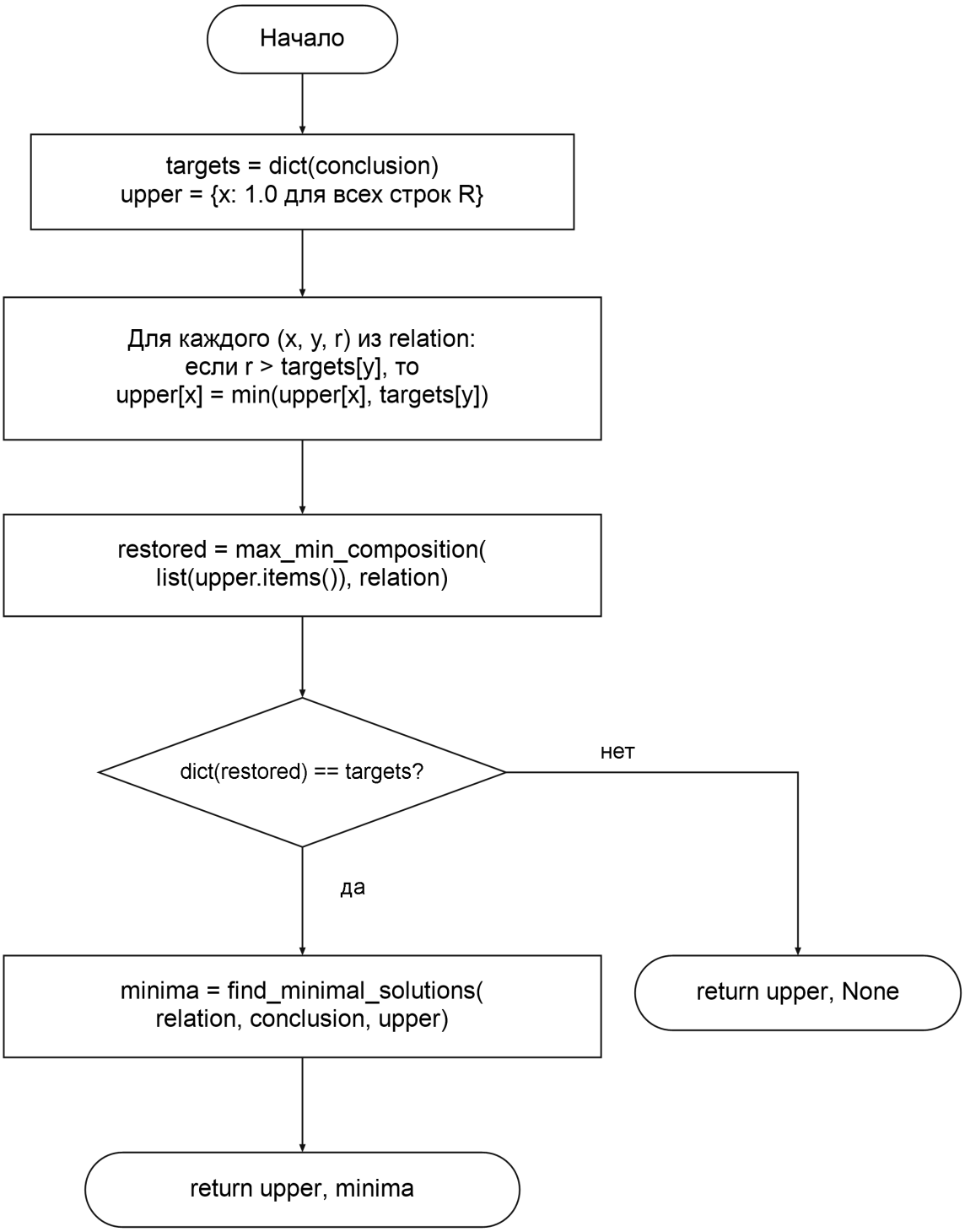


Рис. 3 Схема функции `inverse_fuzzy_inference()`

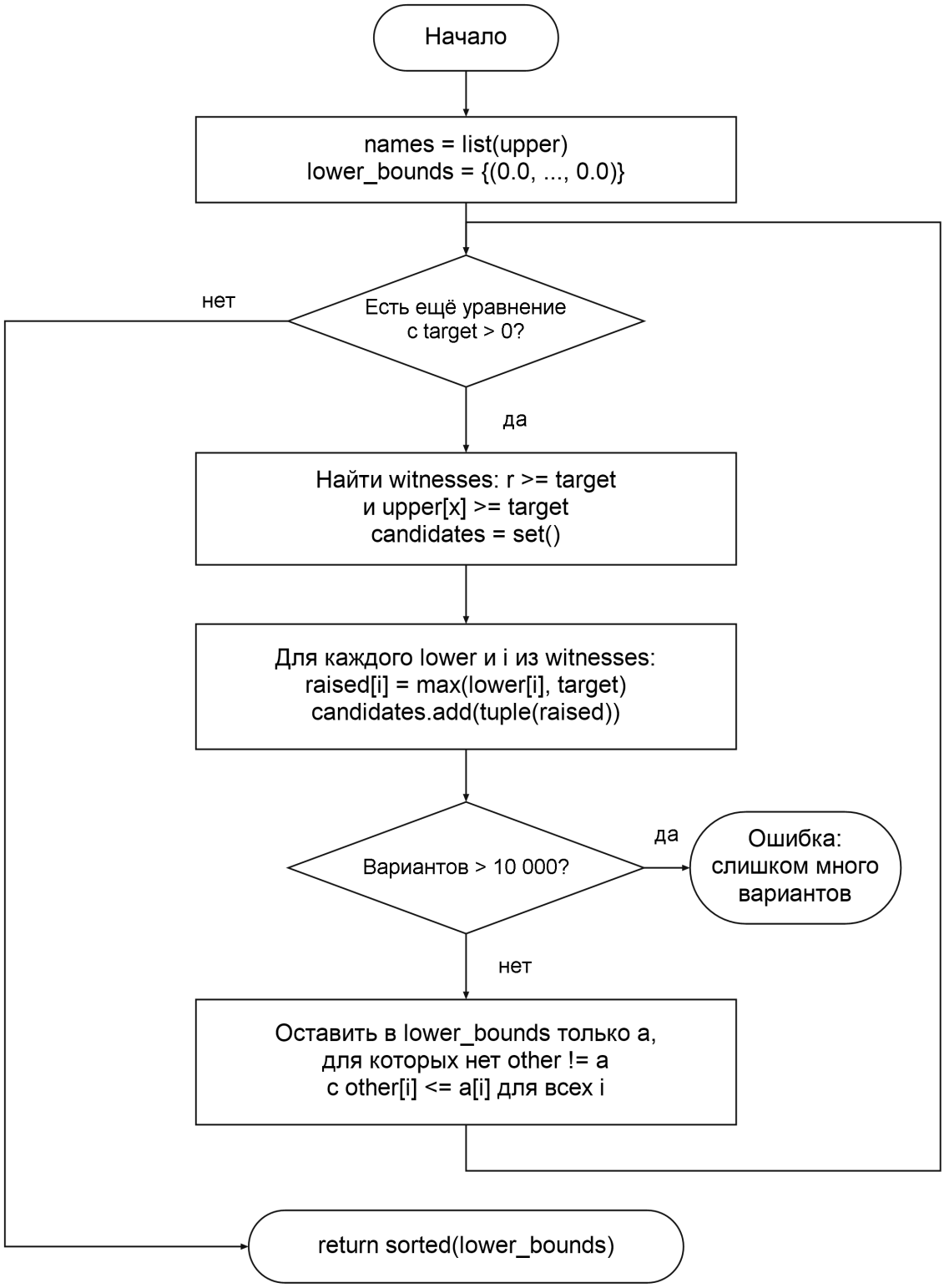


Рис. 4 Схема функции `find_minimal_solutions()`

Демонстрация результатов работы программы

Ниже приведены входные данные и фактические результаты семи запусков `lab4.py`. Авторские комментарии в начале входных файлов для краткости опущены. В выводе `x` означает декартово произведение, `UNION` — объединение областей. Первые строки задают порядок координат, известное следствие, верхнюю границу и её прямую проверку.

Запуск программы без аргументов:

```
python lab4.py
```

Он использует `lab4/input.txt`, совпадающий с первым примером. Для другого файла используется команда `python lab4.py lab4/examples/inputN.txt`.

Полнота результатов проверена независимым перебором 33 777 сочетаний малых матриц, следствий и проверяемых посылок. Для каждой точки принадлежность выданным областям сопоставлена с прямым вычислением уравнений. Дополнительно все семь примеров сопоставлены с предыдущей реализацией и проверена обработка ошибочного ввода.

Пример 1. Правило Гёделя из исходного пояснения. Получается всё множество $[0, 0.4] \times [0.7, 1]$.

Входной файл: `lab4/examples/input1.txt`.

```
A = {<x1, 0.3>, <x2, 0.8>}
B = {<y1, 0.4>, <y2, 0.7>}
C = {<y1, 0.4>, <y2, 0.7>}

A(x) ~> B(y)
```

Результат программы:

```
Порядок координат: x1, x2
Следствие C = {<y1, 0.4>,<y2, 0.7>}
Верхняя граница = {<x1, 0.4>,<x2, 1.0>}
Проверка = {<y1, 0.4>,<y2, 0.7>}
Минимальные решения: [(0.0, 0.7)]
S =
  [0.0, 0.4] x [0.7, 1.0]
Декартово произведение: да
```

Пример 2. Одинаковые столбцы отношения требуют одинаковых компонент следствия. Значения 0.3 и 0.7 несовместны, хотя каждое по отдельности достижимо.

Входной файл: `lab4/examples/input2.txt`.

```
X1 = {<y1, 1.0>, <y2, 1.0>}
X2 = {<y1, 1.0>, <y2, 1.0>}
C = {<y1, 0.3>, <y2, 0.7>}
```

Результат программы:

```
Порядок координат: X1, X2
Следствие C = {<y1, 0.3>,<y2, 0.7>}
Верхняя граница = {<X1, 0.3>,<X2, 0.3>}
Проверка = {<y1, 0.3>,<y2, 0.3>}
Решений нет.
```

Пример 3. Уравнение $\max(a_1, a_2) = 0.5$ имеет два минимальных решения. Пары $(0, 0.5)$ и $(0.5, 0)$ допустимы, а $(0, 0)$ — нет.

Входной файл: `lab4/examples/input3.txt`.

```
A = {<x1, 0.1>, <x2, 0.2>}
B = {<y1, 0.5>}
C = {<y1, 0.5>}

A(x) ~> B(y)
```

Результат программы:

```
Порядок координат: x1, x2
Следствие C = {<y1, 0.5>}
Верхняя граница = {<x1, 0.5>,<x2, 0.5>}
Проверка = {<y1, 0.5>}
Минимальные решения: [(0.0, 0.5), (0.5, 0.0)]
S =
  [0.0, 0.5] x [0.5, 0.5]
  UNION [0.5, 0.5] x [0.0, 0.5]
Декартово произведение: нет
```

Пример 4. Единичная матрица даёт единственное решение $(0.2, 0.8)$. Нижняя и верхняя границы совпадают.

Входной файл: `lab4/examples/input4.txt`.

```
X1 = {<y1, 1.0>, <y2, 0.0>}
X2 = {<y1, 0.0>, <y2, 1.0>}
C = {<y1, 0.2>, <y2, 0.8>}
```

Результат программы:

```
Порядок координат: X1, X2
Следствие C = {<y1, 0.2>,<y2, 0.8>}
Верхняя граница = {<X1, 0.2>,<X2, 0.8>}
Проверка = {<y1, 0.2>,<y2, 0.8>}
Минимальные решения: [(0.2, 0.8)]
S =
  [0.2, 0.2] x [0.8, 0.8]
Декартово произведение: да
```

Пример 5. Нулевое следствие требует $a_1 = 0$, а вторая координата свободна: $a_2 \in [0, 1]$.

Входной файл: `lab4/examples/input5.txt`.

```
X1 = {<y1, 0.0>, <y2, 0.4>}
X2 = {<y1, 0.0>, <y2, 0.0>}
C = {<y1, 0.0>, <y2, 0.0>}
```

Результат программы:

```
Порядок координат: X1, X2
Следствие C = {<y1, 0.0>,<y2, 0.0>}
Верхняя граница = {<X1, 0.0>,<X2, 1.0>}
Проверка = {<y1, 0.0>,<y2, 0.0>}
Минимальные решения: [(0.0, 0.0)]
S =
  [0.0, 0.0] x [0.0, 1.0]
Декартово произведение: да
```

Пример 6. Три неизвестные и два уравнения дают три минимальных вектора и объединение трёх областей.

Входной файл: `lab4/examples/input6.txt`.

```
X1 = {<y1, 0.8>, <y2, 0.2>}
X2 = {<y1, 0.6>, <y2, 0.9>}
X3 = {<y1, 0.3>, <y2, 0.7>}
C = {<y1, 0.6>, <y2, 0.7>}
```

Результат программы:

```
Порядок координат: X1, X2, X3
Следствие C = {<y1, 0.6>,<y2, 0.7>}
Верхняя граница = {<X1, 0.6>,<X2, 0.7>,<X3, 1.0>}
Проверка = {<y1, 0.6>,<y2, 0.7>}
Минимальные решения: [(0.0, 0.6, 0.7), (0.0, 0.7, 0.0), (0.6, 0.0, 0.7)]
S =
  [0.0, 0.6] x [0.6, 0.7] x [0.7, 1.0]
  UNION [0.0, 0.6] x [0.7, 0.7] x [0.0, 1.0]
  UNION [0.6, 0.6] x [0.0, 0.7] x [0.7, 1.0]
Декартово произведение: нет
```

Пример 7. Уравнение $\min(a_1, 0.4) = 0.6$ не имеет решений, поскольку левая часть не превосходит 0.4.

Входной файл: `lab4/examples/input7.txt`.

```
X1 = {<y1, 0.4>}
C = {<y1, 0.6>}
```

Результат программы:

```
Порядок координат: X1
Следствие C = {<y1, 0.6>}
Верхняя граница = {<X1, 1.0>}
Проверка = {<y1, 0.4>}
Решений нет.
```

Ответы на контрольные вопросы

Контрольный вопрос 1

Постановка вопроса. Всегда ли существует решение системы уравнений, заданной правилом и следствием?

Решение. Не всегда. Следствие должно принадлежать образу отображения $a \mapsto a \circ R$. Необходимое и достаточное условие существования точного решения — равенство $u \circ R = C$ для границы (3). Необходимые оценки $c_j \le \max_i r_{ij}$ сами по себе недостаточны: разные уравнения используют общие неизвестные и могут противоречить друг другу.

Пример. При $R = (0.4)$ и $C = (0.6)$ уравнение имеет вид $\min(a_1, 0.4) = 0.6$. Его левая часть не превосходит 0.4. Программа на файле `examples/input7.txt` находит $u = (1)$, но $u \circ R = (0.4) \neq C$ и сообщает об отсутствии решения.

Более сильный пример — $R = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ и $C = (0.3, 0.7)$. Максимум коэффициентов каждого столбца равен 1, но обе компоненты результата равны одному числу $\max(a_1, a_2)$. Поэтому разные правые части несовместны; это проверено в примере 2.

Ответ. Нет. Система совместна ровно тогда, когда кандидат u воспроизводит известное следствие прямой max-min композицией.

Контрольный вопрос 2

Постановка вопроса. При каких правилах и следствии множество наборов значений посылочного предиката не будет декартовым произведением множеств значений для каждого аргумента?

Решение. Рассмотрим непустое множество решений S. Из (5) следует, что его проекции на координатные оси равны интервалам

$$\pi_i(S) = [d_i, u_i], \qquad d_i = \min_{l \in L} l_i.$$

Объединяемые интервалы имеют общую правую границу, поэтому каждая проекция действительно является одним интервалом. Их декартово произведение равно $[d, u]$ и в общем случае содержит больше точек, чем S.

Если d является решением, то весь интервал $[d, u]$ состоит из решений и совпадает с S. Если S декартово, его нижний угол d обязан принадлежать S. Поэтому

$$S = \prod_i \pi_i(S) \quad \Longleftrightarrow \quad d \circ R = C \quad \Longleftrightarrow \quad |L| = 1. \tag{6}$$

Последняя эквивалентность следует из минимальности: два различных минимальных вектора несравнимы. Их покоординатный минимум не может быть решением, иначе они не были бы минимальными.

Таким образом, множество недекартово, когда положительные уравнения допускают несколько несравнимых минимальных способов достижения требуемых максимумов. Самого наличия нескольких кандидатов в одном J_j недостаточно: другие уравнения могут принудительно выбрать одного из них.

Пример. Для $R = \binom{1}{1}$ и $C = (0.5)$ получаем

$$S = \big(\{0.5\} \times [0, 0.5]\big) \cup \big([0, 0.5] \times \{0.5\}\big).$$

Обе проекции равны $[0, 0.5]$, но $(0, 0)$ из их произведения не удовлетворяет уравнению $\max(a_1, a_2) = 0.5$. Программа находит два минимальных решения $(0.5, 0)$ и $(0, 0.5)$.

Ответ. При $S \neq \varnothing$ множество недекартово ровно тогда, когда $|L| > 1$. Эквивалентная проверка: $d \circ R \neq C$.

Контрольный вопрос 3

Постановка вопроса. Какое минимальное количество переменных необходимо при выполнении условий предыдущего вопроса?

Решение. При одной неизвестной каждое уравнение имеет вид $\min(a_1, r_{1j}) = c_j$. Если $r_{1j} < c_j$, решений нет. Если $r_{1j} = c_j$, допустим интервал $[c_j, 1]$. Если $r_{1j} > c_j$, допустима единственная точка $a_1 = c_j$. Пересечение таких ограничений либо пусто, либо является интервалом, включая одноточечный. Для одной координаты непустое множество решений всегда совпадает со своей единственной проекцией.

Уже для двух неизвестных может возникнуть выбор между независимыми способами достижения максимума. Достаточно одного уравнения

$$\max\{\min(a_1, 1), \min(a_2, 1)\} = 0.5.$$

Его множество решений приведено в ответе на второй вопрос и не является декартовым произведением.

Пример. Отношение из двух единиц получается и из правила первой лабораторной: $A = (0.1, 0.2)$, $B = (0.5)$, поскольку обе степени A не превосходят 0.5. При наблюдаемом следствии $C = (0.5)$ программа на файле `examples/input3.txt` находит два минимальных решения.

Ответ. Минимально нужны две неизвестные степени принадлежности посылки, то есть два аргумента посылочного предиката. При этом достаточно одного элемента следствия и одного уравнения.

Личный вклад

В работе использованы части реализации первой лабораторной со следующим распределением:

Сайковский А. В.: max-min композиция и механизм логического вывода.

Мяло В. В.: реализация импликации Гёделя.

Вашкевич М. В.: регулярные выражения и обработка строк базы знаний.

Обратный вывод, его входные проверки и форматирование результата объединены в файле `lab4.py`.

Вывод

На основе первой лабораторной работы разработана программа обратного нечёткого логического вывода для max-min композиции. Сохранены исходный формат нечётких множеств и функции чтения данных, построения отношения, прямой композиции и форматирования. Дополнительный код размещён в одном файле.

Реализованы вычисление верхней границы, проверка существования решения и поиск всех минимальных посылок. Множество решений описывается объединением замкнутых декартовых областей. На примерах проверены совместная и несовместная системы, единственное решение, свободная координата и недекартово множество допустимых посылок.

Показано, что решение существует не всегда; покоординатные диапазоны нельзя без проверки объединять в одно произведение. Для появления недекартова множества достаточно двух неизвестных. Подготовлены редактируемые блок-схемы draw.io и результаты запусков программы.

Листинг кода

Файл `lab4.py`. Импортируемые функции первой лабораторной используются напрямую из `lab1.py`.

```
# Лабораторная работа №4 (индивидуальная практическая работа 2), ЛОИС.
# Выполнена студентами группы 421702 БГУИР:
# Сайковский А. В., Мяло В. В., Вашкевич М. В.
# Обратный max-min вывод: все посылки, дающие известное следствие.
# Версия 2.0, 12.11.2026: один файл и прямое использование первой лабы.
# Источники: авторы lab1.py; Голенков В. В. и др., «ЛОИС. Практикум», 2011;
# E. Sanchez, https://doi.org/10.1016/S0019-9958(76)90446-0;
# Python Software Foundation, https://docs.python.org/3/library/.

import sys
from pathlib import Path

from lab1 import (
    FACT_PATTERN, RULE_PATTERN, read_knowledge_base, build_godel_relation,
    max_min_composition, format_fuzzy_set,
)


def read_problem(filename):
    """C - известное следствие; одно правило или факты-строки матрицы."""
    content = Path(filename).read_text(encoding="utf-8")
    for line in content.splitlines():
        if line.strip() and not line.lstrip().startswith("#"):
            if not (FACT_PATTERN.fullmatch(line) or
             ↪  RULE_PATTERN.fullmatch(line)):
                raise ValueError(f"Неверная строка: {line}")
    facts, rules = read_knowledge_base(filename)
    if "C" not in facts or len(rules) > 1:
        raise ValueError("Нужен факт C и не более одного правила")
    if len(list(FACT_PATTERN.finditer(content))) != len(facts):
        raise ValueError("Имена фактов не должны повторяться")
    if any(len(dict(pairs)) != len(pairs) or not pairs for pairs in
     ↪  facts.values()):
        raise ValueError("Нужны непустые множества без повторов элементов")
    if rules:
        left, right = rules[0]
        if left not in facts or right not in facts or "C" in (left, right):
            raise ValueError("Правило должно ссылаться на факты, отличные от
             ↪  C")
        relation = build_godel_relation(facts[left], facts[right])
    else:
        relation = [(x, y, r) for x, row in facts.items() if x != "C" for y,
         ↪  r in row]
    if not relation:
        raise ValueError("Отношение не задано")
    domain = set(dict(facts["C"]))
    for x in dict.fromkeys(x for x, _, _ in relation):
```

```
        if {y for source, y, _ in relation if source == x} != domain:
            raise ValueError("Каждая строка отношения должна иметь тот же
            ↪  универсум, что C")
    return relation, facts["C"]


def find_minimal_solutions(relation, conclusion, upper):
    """Выбрать свидетелей максимумов и оставить минимальные нижние
    ↪  границы."""
    names = list(upper)
    lower_bounds = {(0.0,) * len(names)}
    for y, target in conclusion:
        if target == 0:
            continue
        witnesses = [names.index(x) for x, column, r in relation
                     if column == y and r >= target and upper[x] >= target]
        candidates = set()
        for lower in lower_bounds:
            for i in witnesses:
                raised = list(lower)
                raised[i] = max(raised[i], target)
                candidates.add(tuple(raised))
                if len(candidates) > 10000:
                    raise ValueError("Слишком много вариантов; полное
                    ↪  множество не вычислено")
        lower_bounds = {a for a in candidates if not any(
            other != a and all(v <= w for v, w in zip(other, a))
            for other in candidates
        )}
    return sorted(lower_bounds)


def inverse_fuzzy_inference(relation, conclusion):
    """Вернуть верхнюю границу и все минимальные решения, либо None."""
    targets = dict(conclusion)
    upper = dict.fromkeys((x for x, _, _ in relation), 1.0)
    for x, y, r in relation:
        if r > targets[y]:
            upper[x] = min(upper[x], targets[y])
    # Проверка существования решения - прямой вывод из первой лабы.
    restored = max_min_composition(list(upper.items()), relation)
    if dict(restored) != targets:
        return upper, None
    return upper, find_minimal_solutions(relation, conclusion, upper)


def format_result(relation, conclusion, upper, minima):
    """S - объединение областей [минимальное решение, верхняя граница]."""
    lines = ["Порядок координат: " + ", ".join(upper),
             "Следствие C = " + format_fuzzy_set(conclusion),
             "Верхняя граница = " + format_fuzzy_set(upper.items()),
```

```
            "Проверка = " +
             ↪  format_fuzzy_set(max_min_composition(list(upper.items()),
             ↪  relation))]
    if minima is None:
        return "\n".join(lines + ["Решений нет."])
    lines.append("Минимальные решения: " + str(minima))
    lines.append("S =")
    for index, lower in enumerate(minima):
        box = " x ".join(f"[{lo}, {hi}]" for lo, hi in zip(lower,
         ↪  upper.values()))
        lines.append(("  " if index == 0 else "  UNION ") + box)
    lines.append("Декартово произведение: " + ("да" if len(minima) == 1 else
     ↪  "нет"))
    return "\n".join(lines)


def main():
    filename = sys.argv[1] if len(sys.argv) > 1 else
     ↪  Path(__file__).with_name("lab4") / "input.txt"
    relation, conclusion = read_problem(filename)
    upper, minima = inverse_fuzzy_inference(relation, conclusion)
    print(format_result(relation, conclusion, upper, minima))


if __name__ == "__main__":
    # Одинаковая кодировка консоли, перенаправления вывода и файлов отчёта.
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    try:
        main()
    except (ValueError, OSError) as error:
        print(f"Ошибка: {error}", file=sys.stderr)
        raise SystemExit(2)
```

Использованные источники

1. Логические основы интеллектуальных систем. Практикум : учебно-методическое пособие / В. В. Голенков, В. П. Ивашенко, Д. Г. Колб, К. А. Уваров. — Минск : БГУИР, 2011. — 70 с. Раздел об обратном нечётком выводе, с. 66–67. URL: `https://libeldoc.bsuir.by/bitstream/123456789/992/2/Log_Osn.pdf` (дата обращения: 12.11.2026).

2. Sanchez E. Resolution of composite fuzzy relation equations // Information and Control. — 1976. — Vol. 30, no. 1. — P. 38–48. DOI: 10.1016/S0019-9958(76)90446-0.

3. Python Software Foundation. Python 3 Documentation: `re`, `pathlib`. URL: `https://docs.python.org/3/library/re.html`; `https://docs.python.org/3/library/pathlib.html` (дата обращения: 12.11.2026).

4. Сайковский А. В., Мяло В. В., Вашкевич М. В. Лабораторная работа № 1 по ЛОИС. Файлы проекта `lab1.py`, `report.tex`, `figures/drawio/`. БГУИР, группа 421702, 2026.